Section 7: Priors, Posteriors and MCMC

GOVT 6039 | Fall 2026 | Discussion section, 21 October 2026

Table of contents

Getting this running 1

Part 1: what a prior is worth **2**

Part 2: approximating the posterior on a grid **6**

Part 3: Metropolis, by hand and in R **9**

Part 4: separation, on real data **14**

Part 5: the real result, exit ticket and Thursday **22**

Part 6: take-home, a Stan check **24**

References **25**

Lecture 13 estimates one proportion, π, from a small sample (12 of 15 Hainan respondents trust local government), moves from the sample share to a Bayesian posterior, and uses Metropolis sampling once a model has no closed-form posterior. In section we run the same machinery on a different question: **in the spring of 1988, what share of Chileans planned to vote to keep General Pinochet in power?**

The data are a national survey run by FLACSO/Chile in April and May 1988, five to six months before the plebiscite of 5 October 1988, in which voters were asked whether Pinochet should stay for another eight years ("Yes") or whether there should be open elections ("No"). The official result was 44.01% Yes and 55.99% No, so unlike the Hainan survey we can check our estimate against what happened. Parts 1 to 3 and 5 use this survey. Part 4 switches to a second real dataset, because the Chile survey has no separated cell (every region has both Yes and No answers) and we want to see the lecture's last problem on real data.

Getting this running

This page is rendered from `section07.qmd`, so if you change the code you change the document.

```
git clone --filter=blob:none --sparse
 ↪  https://github.com/ddekadt/cornell-govt-6039.git
cd cornell-govt-6039
git sparse-checkout set sections/section07
cd sections/section07
quarto render section07.qmd
```

The R packages are `ggplot2` and `patchwork`. Part 4 compares against Firth's penalized likelihood if `brglm2` is installed (optional), and Part 6 uses `rstanarm` (take-home). The data are small enough to be typed into the file, so nothing needs downloading; the full respondent-level files are `Chile` and `Greene` in the `carData` package (Fox 2016).

Part 1: what a prior is worth

The survey asks how each respondent will vote. We keep the people who gave a Yes or No intention ("decided") and let π be the share of them who say **Yes**, that is, who want Pinochet to stay. The counts by region are

```
chile <- read.csv(text = "region,A,N,U,Y
C,44,210,141,174
M,2,18,23,38
N,30,102,46,135
S,42,214,148,275
SA,69,345,230,246")      # A abstain, N no, U undecided, Y yes (carData::Chile,
 ↪  vote by region)

chile$decided <- chile$Y + chile$N
chile$share   <- chile$Y / chile$decided

rM <- chile$Y[chile$region == "M"];  nM <- chile$decided[chile$region == "M"]
 ↪  # Metropolitan Santiago area
rS <- chile$Y[chile$region == "SA"]; nS <- chile$decided[chile$region == "SA"]
 ↪  # city of Santiago
```

Region	Description	Decided (n)	Yes (r)	Sample share
C	Central	384	174	0.453
M	Metropolitan Santiago area	56	38	0.679
N	North	237	135	0.570
S	South	489	275	0.562
SA	City of Santiago	591	246	0.416

Region M plays the part of Hainan (a small sample with a high sample share) and SA the part of Guangdong (a large sample). The figure shows how far each region's sample share can be trusted on its own:

```r
reg <- chile
reg$lo  <- qbeta(0.025, 1 + reg$Y, 1 + reg$decided - reg$Y)
reg$hi  <- qbeta(0.975, 1 + reg$Y, 1 + reg$decided - reg$Y)
reg$lab <- sprintf("%s  %s (n = %d)", reg$region, reg_lab[reg$region],
 ↪  reg$decided)
reg$lab <- factor(reg$lab, levels = reg$lab[order(reg$share)])
reg$hl  <- ifelse(reg$region %in% c("M", "SA"), reg$region, "other")

ggplot(reg, aes(share, lab, colour = hl)) +
  geom_vline(xintercept = sum(chile$Y) / sum(chile$decided), linetype = "dashed",
   ↪  colour = ref) +
  geom_linerange(aes(xmin = lo, xmax = hi), linewidth = 1.6, alpha = 0.55) +
  geom_point(size = 3) +
  scale_colour_manual(values = c(M = pal[["orange"]], SA = pal[["blue"]], other =
   ↪  ref)) +
  scale_x_continuous(limits = c(0.3, 0.85), breaks = seq(0.3, 0.8, 0.1)) +
  labs(title = "Where region M's estimate is least certain",
       subtitle = "Dashed line: all regions pooled", x = "Share saying Yes", y =
        ↪  NULL)
```

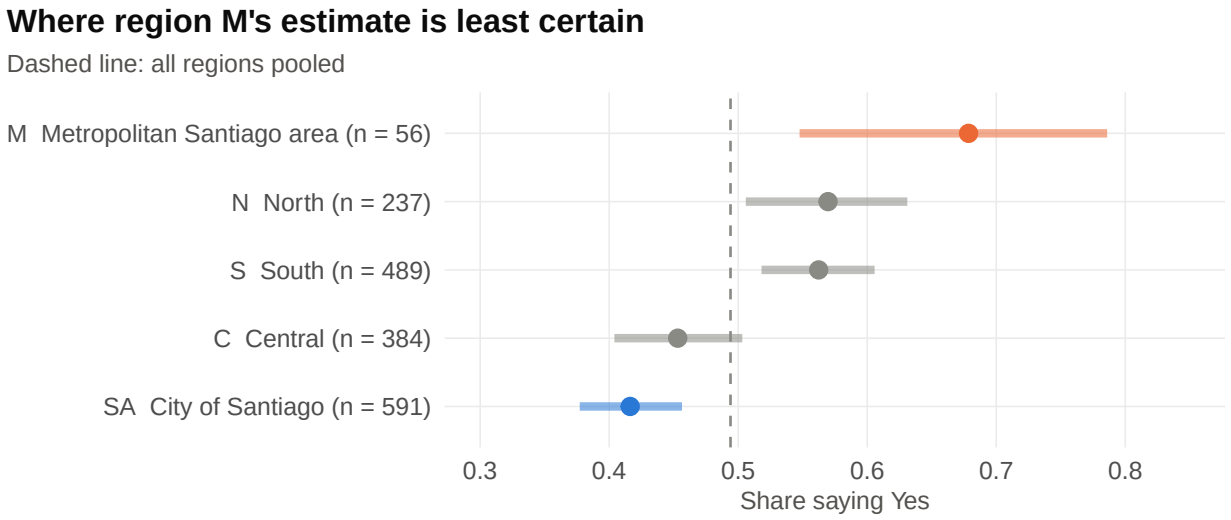


Figure 1: Sample share of Yes by region with 95% intervals under a flat Beta(1, 1) prior. Region M's interval is wide because it rests on 56 respondents.

1.1 Use the prior Beta(10, 10): mean $10/(10+10) = 0.5$, "worth 20 respondents". Write the posterior mean of π as a weighted average of the prior mean and the sample share. For M and for SA, how much weight does the prior get, and what is the posterior mean?

1.2 Now a stronger prior, Beta(50, 50): same centre, worth 100 respondents. *Before calculating*, say which region's estimate moves more away from its sample share, and roughly how much weight the prior gets in each. Then compute both posterior means. The chunk below draws prior, likelihood and posterior for M and SA under both priors, so you can check your prediction against it.

```r
px <- seq(0.15, 0.95, length.out = 400)
curves <- function(label, r, n, a, b) {
  data.frame(p = px, region = label, prior = sprintf("Prior Beta(%d, %d)", a, b),
             curve = factor(rep(c("Prior", "Likelihood (rescaled)", "Posterior"),
              ↪  each = length(px)),
                            levels = c("Prior", "Likelihood (rescaled)",
                             ↪  "Posterior")),
             d = c(dbeta(px, a, b), dbeta(px, r + 1, n - r + 1), dbeta(px, a + r,
              ↪  b + n - r)))
}
lab_M <- sprintf("M: %d of %d", rM, nM); lab_S <- sprintf("SA: %d of %d", rS, nS)
d_worth <- rbind(curves(lab_M, rM, nM, 10, 10), curves(lab_M, rM, nM, 50, 50),
                 curves(lab_S, rS, nS, 10, 10), curves(lab_S, rS, nS, 50, 50))
d_worth$region <- factor(d_worth$region, levels = c(lab_M, lab_S))

ggplot(d_worth, aes(p, d, colour = curve, linetype = curve, linewidth = curve)) +
  geom_line() +
  facet_grid(region ~ prior, scales = "free_y") +
  scale_colour_manual(values = c(Prior = ref, `Likelihood (rescaled)` =
   ↪  pal[["orange"]], Posterior = pal[["blue"]]), name = NULL) +
  scale_linetype_manual(values = c(Prior = "dashed", `Likelihood (rescaled)` =
   ↪  "solid", Posterior = "solid"), name = NULL) +
  scale_linewidth_manual(values = c(Prior = 0.7, `Likelihood (rescaled)` = 0.7,
   ↪  Posterior = 1.2), name = NULL) +
  theme(legend.position = "bottom", strip.text = element_text(colour = ink, face
   ↪  = "bold")) +
  labs(x = expression(pi), y = "Density")
```

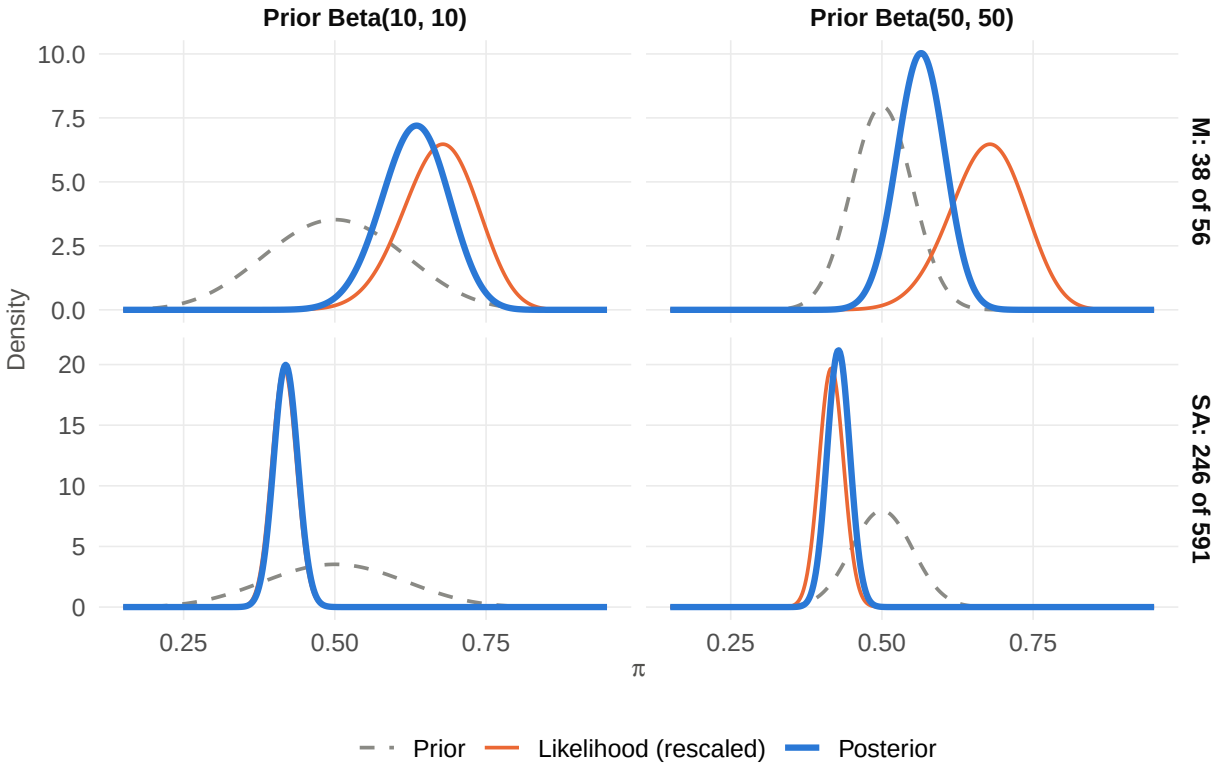


Figure 2: Prior, likelihood (rescaled to integrate to one) and posterior for M and SA under a Beta(10, 10) and a Beta(50, 50) prior. Rows have their own vertical scales.

1.3 *(If time.)* A prior has to come from somewhere. Two candidates for M:

- **(a)** The other four regions' sample shares, which have the centre and standard deviation printed below. A Beta(a, b) has mean $m = a/(a+b)$ and variance $m(1-m)/(a+b+1)$, so a Beta that matches a centre m and a sd s has $a + b = m(1-m)/s^2 - 1$.
- **(b)** The official result of Chile's 1980 constitutional plebiscite, about 68% Yes, with the weight of 20 respondents: Beta(13.6, 6.4).

```
other <- subset(chile, region != "M")
c(centre = mean(other$share), sd = sd(other$share))
```

```
    centre         sd
0.50034027 0.07735028
```

```
px <- seq(0.15, 0.95, length.out = 400)
prior_levels <- c("Beta(20, 20): other regions", "Beta(10, 10)", "Beta(50, 50)",
 ↪  "Beta(13.6, 6.4): 1980 vote")
d_pri <- data.frame(p = rep(px, 4), prior = factor(rep(prior_levels, each =
 ↪  length(px)), levels = prior_levels),
                    d = c(dbeta(px, 20, 20), dbeta(px, 10, 10), dbeta(px, 50,
                     ↪  50), dbeta(px, 13.6, 6.4)))
```

```
ggplot(d_pri, aes(p, d, colour = prior, linetype = prior, linewidth = prior)) +
  geom_line() +
  geom_vline(xintercept = rM / nM, colour = pal[["orange"]], linewidth = 0.7) +
  annotate("text", x = rM / nM + 0.012, y = 9.3, label = "M's sample share",
   ↪  hjust = 0, colour = pal[["orange"]], size = 3.4) +
  geom_point(data = data.frame(p = other$share, d = 0), aes(p, d), inherit.aes =
   ↪  FALSE,
             shape = 21, size = 3.2, fill = "white", colour = ink) +
  scale_colour_manual(values = c(pal[["blue"]], pal[["aqua"]], ref, ink2), name =
   ↪  NULL) +
  scale_linetype_manual(values = c("solid", "solid", "dashed", "dotdash"), name =
   ↪  NULL) +
  scale_linewidth_manual(values = c(1.2, 0.8, 0.8, 0.8), name = NULL) +
  theme(legend.position = "right") +
  labs(x = expression(pi), y = "Density")
```

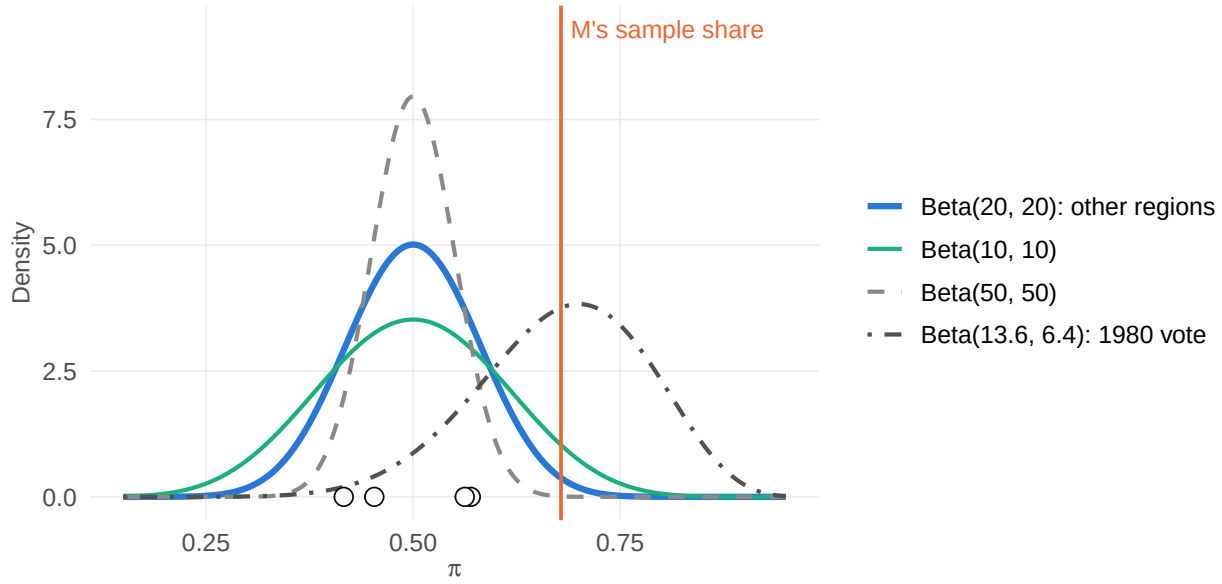


Figure 3: Candidate priors for region M. The open circles are the four other regions' sample shares; the orange line is M's own sample share.

Find the Beta that matches (a). Are Beta(10, 10) and Beta(50, 50) fair roundings of it? Compute M's posterior mean under (a) and under (b). Which prior would you defend?

Part 2: approximating the posterior on a grid

For a handful of guesses of π, the posterior is prior $\times$ likelihood at each guess, divided by their sum. We estimate π for region M, with the prior Beta(20, 20) and five guesses:

```
r <- rM; n <- nM                                    # Region M: 38 of 56 decided
 ↪  respondents say Yes

grid_post <- function(p, a = 20, b = 20) {
  w <- dbeta(p, a, b) * p^r * (1 - p)^(n - r)      # prior x likelihood at each
 ↪  guess
  w / sum(w)                                        # divide by the sum: the
 ↪  posterior on the grid
}

p5      <- c(0.4, 0.5, 0.6, 0.7, 0.8)
p5_post <- grid_post(p5)
round(data.frame(p = p5, posterior = p5_post), 2)
```

```
    p posterior
1 0.4      0.00
2 0.5      0.09
3 0.6      0.78
4 0.7      0.12
5 0.8      0.00
```

2.1 Where does the posterior peak? The grid point closest to the sample share of 0.679 is 0.7. Why is the peak not there?

Now refine to 1,000 guesses and summarize the posterior three ways: the five-point grid, the 1,000-point grid, and the exact Beta(58, 38).

```
p_fine <- seq(0.0005, 0.9995, length.out = 1000)
w_fine <- grid_post(p_fine)

grid_summary <- function(p, w) {
  c(mean     = sum(p * w),
    lo       = p[which(cumsum(w) >= 0.025)[1]],
    hi       = p[which(cumsum(w) >= 0.975)[1]],
    p_gt_half = sum(w[p > 0.5]))
}

a1 <- 20 + rM; b1 <- 20 + nM - rM                   # conjugacy: Beta(20 + r, 20 +
 ↪  n - r) = Beta(58, 38)
tab2 <- rbind(`5 points`           = grid_summary(p5, p5_post),
              `1,000 points`       = grid_summary(p_fine, w_fine),
              `Exact Beta(58, 38)` = c(a1 / (a1 + b1), qbeta(0.025, a1, b1),
               ↪  qbeta(0.975, a1, b1), 1 - pbeta(0.5, a1, b1)))
knitr::kable(tab2, digits = 3, col.names = c("Mean", "2.5%", "97.5%", "P(pi >
 ↪  0.5)"))
```

	Mean	2.5%	97.5%	P(pi > 0.5)
5 points	0.603	0.500	0.700	0.907
1,000 points	0.604	0.505	0.700	0.980
Exact Beta(58, 38)	0.604	0.505	0.699	0.980

```
px    <- seq(0.15, 0.95, length.out = 400)
exact <- data.frame(p = px, d = dbeta(px, a1, b1))
g_five <- ggplot() +
  geom_col(data = data.frame(p = p5, d = p5_post / 0.1), aes(p, d), width =
   ↪  0.085, fill = pal[["blue"]], alpha = 0.45) +
  geom_line(data = exact, aes(p, d), colour = pal[["orange"]], linewidth = 0.9,
   ↪  linetype = "dashed") +
  coord_cartesian(xlim = c(0.3, 0.9)) + labs(title = "5 guesses", x =
   ↪  expression(pi), y = "Posterior density")
g_fine <- ggplot() +
  geom_point(data = data.frame(p = p_fine, d = w_fine / (p_fine[2] - p_fine[1])),
   ↪  aes(p, d),
             colour = pal[["blue"]], size = 0.8, alpha = 0.6) +
  geom_line(data = exact, aes(p, d), colour = pal[["orange"]], linewidth = 0.9,
   ↪  linetype = "dashed") +
  coord_cartesian(xlim = c(0.3, 0.9)) + labs(title = "1,000 guesses", x =
   ↪  expression(pi), y = NULL)
g_five + g_fine
```

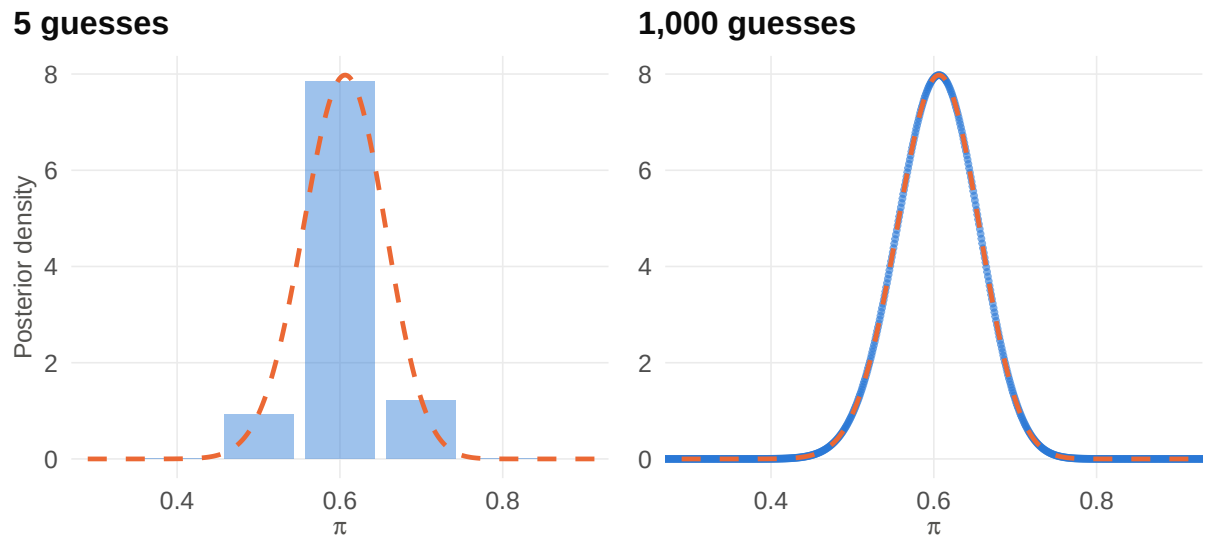


Figure 4: The grid posterior (blue) against the exact Beta(58, 38) density (orange, dashed). Grid weights are divided by the spacing so that they sit on the density scale.

2.2 How close is each grid to the exact answer? What goes wrong with five points?

2.3 Slide 15 says 10^8 evaluations take 1.1 hours. **(a)** What speed is that? **(b)** Check that 10^{14} evaluations take about 127 years. **(c)** A regression with 20 coefficients and 100 guesses per coefficient: how long? **(d)** Why is most of that work wasted?

Part 3: Metropolis, by hand and in R

3.1 Region M, prior Beta(20, 20), steps from $N(0, 0.1^2)$. Three snapshots, each with the chain currently at $\pi^{(t)} = 0.550$:

Case	$\pi^{(t)}$	π^*	u
A	0.550	0.470	0.300
B	0.550	0.500	0.150
C	0.550	0.620	0.810

(a) Multiply the prior kernel $\pi^{19}(1-\pi)^{19}$ by the likelihood $\pi^{38}(1-\pi)^{18}$. Which known distribution is this the kernel of? **(b)** Compute the ratio for each case. **(c)** The chain moves if $u <$ ratio. Does it move in each case? **(d)** What happened to the normalizing constant $p(Y)$?

The figure shows where each case sits on the posterior. The dotted lines mark the heights that the ratio compares.

```
cases  <- data.frame(case = c("A", "B", "C"), cur = 0.55, prop = c(0.47, 0.50,
 ↪  0.62))
px     <- seq(0.3, 0.8, length.out = 300)
curve_d <- merge(data.frame(p = px, d = dbeta(px, a1, b1)), data.frame(case =
 ↪  cases$case))
pts <- do.call(rbind, lapply(seq_len(nrow(cases)), function(i) data.frame(
  case = cases$case[i], p = c(cases$cur[i], cases$prop[i]), which = c("current",
   ↪  "proposal"),
  d = dbeta(c(cases$cur[i], cases$prop[i]), a1, b1))))

ggplot(curve_d, aes(p, d)) +
  geom_area(fill = pal[["blue"]], alpha = 0.12) + geom_line(colour =
   ↪  pal[["blue"]], linewidth = 0.9) +
  geom_segment(data = pts, aes(x = p, xend = p, y = 0, yend = d, colour = which),
   ↪  linetype = "dotted", linewidth = 0.6) +
  geom_point(data = pts, aes(colour = which, shape = which), size = 3.6) +
  scale_colour_manual(values = c(current = ink, proposal = pal[["orange"]]), name
   ↪  = NULL) +
  scale_shape_manual(values = c(current = 16, proposal = 17), name = NULL) +
  facet_wrap(~ case, nrow = 1, labeller = labeller(case = function(x)
   ↪  paste("Case", x))) +
  theme(legend.position = "bottom", strip.text = element_text(colour = ink, face
   ↪  = "bold")) +
  labs(x = expression(pi), y = "Posterior density")
```

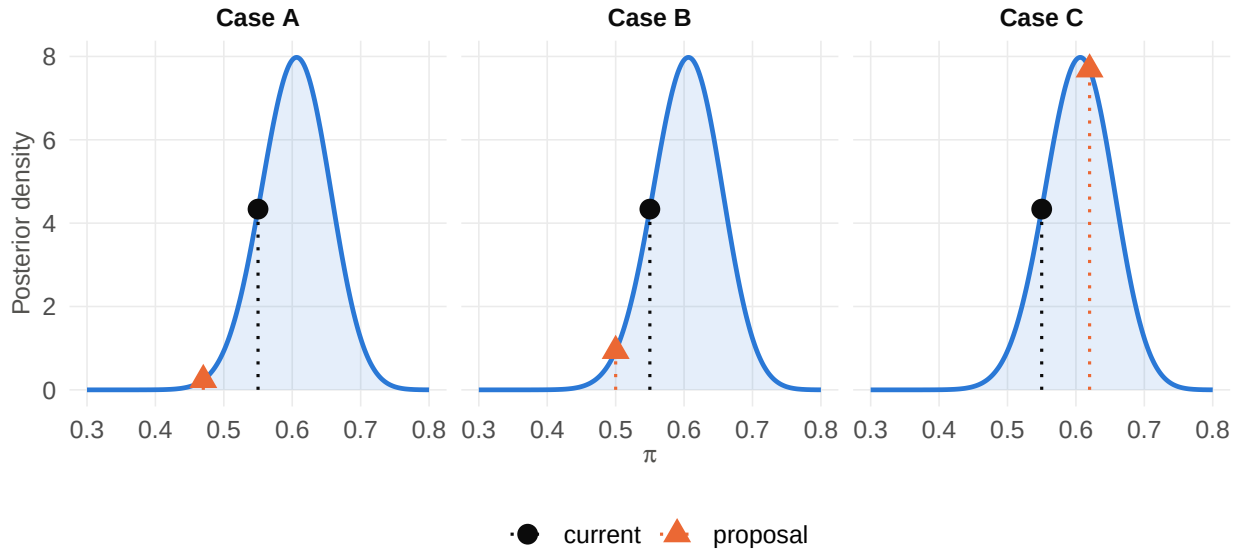


Figure 5: The three snapshots of 3.1 on the exact posterior Beta(58, 38): the current value (circle) and the proposal (triangle).

Here is the whole algorithm, three short functions:

```
log_post <- function(p, r = rM, n = nM, a = 20, b = 20) {
  # outside (0, 1) the prior density is 0, so the log is -Inf
  if (p <= 0 || p >= 1) return(-Inf)
  # log(likelihood x prior), up to a constant
  r * log(p) + (n - r) * log(1 - p) + dbeta(p, a, b, log = TRUE)
}

metropolis <- function(start, n_iter, step_sd, ...) {
  draws <- numeric(n_iter); cur <- start
  for (t in seq_len(n_iter)) {
    # propose a random step
    prop      <- cur + rnorm(1, 0, step_sd)
    # log of the ratio of (prior x likelihood) at the proposal and at the current
     ↪  value
    log_ratio <- log_post(prop, ...) - log_post(cur, ...)
    # move, or stay
    if (log(runif(1)) < log_ratio) cur <- prop
    # a rejected step records the current value again
    draws[t] <- cur
  }
  draws
}

# R-hat; m is an (iterations x chains) matrix
```

```
rhat <- function(m) {
  n <- nrow(m); W <- mean(apply(m, 2, var)); B_n <- var(colMeans(m))
  sqrt(((n - 1) / n * W + B_n) / W)
}
# share of steps on which the chain moved
acc_rate <- function(m) mean(apply(m, 2, function(x) mean(diff(x) != 0)))
```

3.2 Read `log_post()` and `metropolis()`. **(a)** Why is `log(runif(1)) < log_ratio` the same as "move with probability $\min(1, \text{ratio})$"? **(b)** What does `return(-Inf)` do when a proposal lands outside $(0, 1)$? **(c)** Why work with logs?

Run four chains from different starting values, drop the first 1,000 draws of each, and compare with the exact Beta(58, 38):

```
set.seed(6039)
starts <- c(0.2, 0.4, 0.8, 0.95)
n_iter <- 20000; burn <- 1000

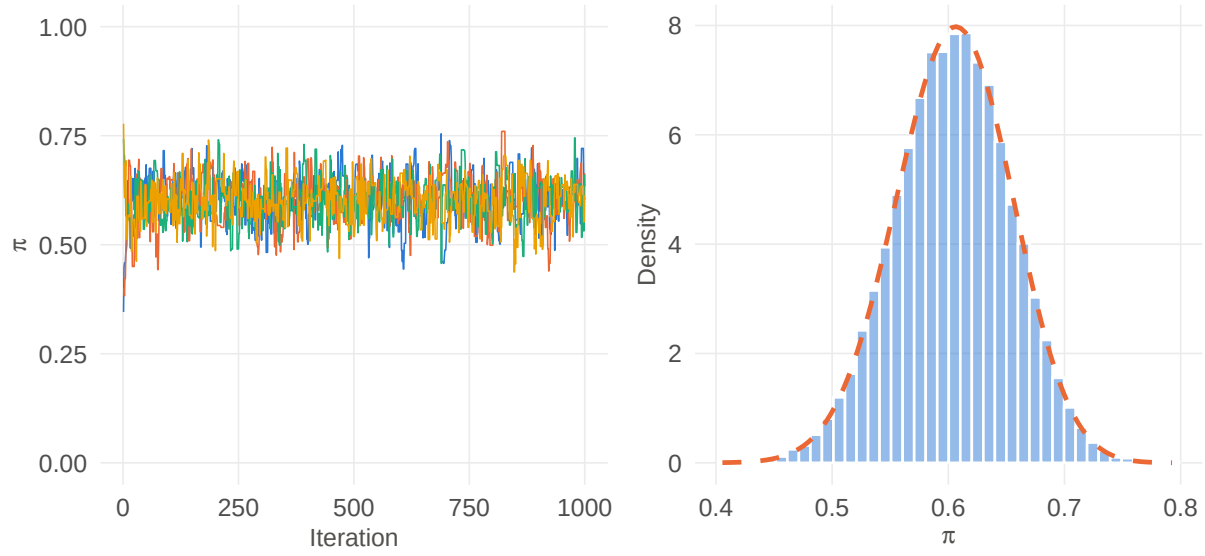
chains <- sapply(starts, function(s) metropolis(s, n_iter, step_sd = 0.1))   #
 ↪  n_iter x 4 matrix
kept   <- chains[-(1:burn), ]
draws  <- as.vector(kept)

draw_tab <- c(mean = mean(draws), lo = unname(quantile(draws, 0.025)), hi =
 ↪  unname(quantile(draws, 0.975)),
              p_gt_half = mean(draws > 0.5), accept = acc_rate(chains), Rhat =
               ↪  rhat(kept))
round(draw_tab, 3)
```

```
     mean        lo        hi p_gt_half    accept      Rhat
    0.604     0.504     0.699     0.980     0.498     1.000
```

```
trace_plot <- function(m, title = NULL, t_max = nrow(m)) {
  d <- data.frame(t = rep(seq_len(t_max), ncol(m)), p =
 ↪  as.vector(m[seq_len(t_max), ]),
                  chain = factor(rep(seq_len(ncol(m)), each = t_max)))
  ggplot(d, aes(t, p, colour = chain)) + geom_line(linewidth = 0.3) +
    scale_colour_manual(values = unname(pal)) + ylim(0, 1) + labs(title = title,
     ↪  x = "Iteration", y = expression(pi))
}
p_hist <- ggplot(data.frame(p = draws), aes(p)) +
  geom_histogram(aes(y = after_stat(density)), bins = 40, fill = pal[["blue"]],
   ↪  alpha = 0.5, colour = "white") +
  stat_function(fun = dbeta, args = list(shape1 = a1, shape2 = b1), colour =
   ↪  pal[["orange"]],
                linewidth = 0.9, linetype = "dashed") +
```

```
  labs(x = expression(pi), y = "Density")
trace_plot(chains, t_max = 1000) + p_hist
```



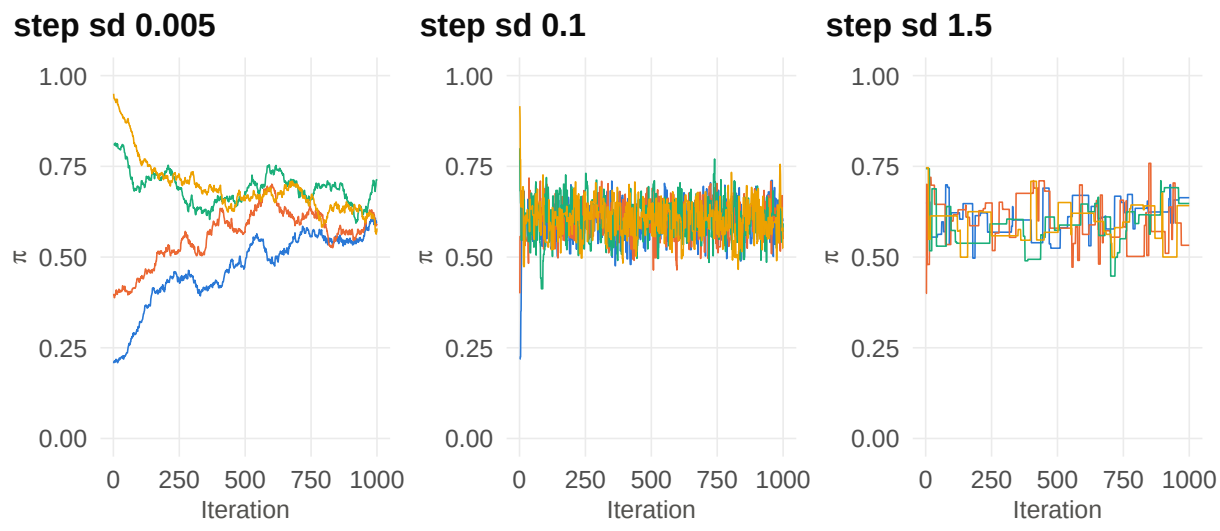
3.3 Compare the mean, interval and $P(\pi > 0.5)$ with the exact Beta(58, 38) values from 2.2. Are they equal? Why or why not?

3.4 *Predict first.* We rerun with step sizes 0.005, 0.1 and 1.5, for 1,000 iterations each. Which will accept the most proposals? Which will have converged after 1,000 iterations? Then run the chunk, and the second chunk, which asks how many independent draws each setting is really worth (the effective sample size).

```
set.seed(6039)
steps    <- c(0.005, 0.1, 1.5)
runs     <- lapply(steps, function(s) sapply(starts, function(st) metropolis(st,
 ↪  1000, step_sd = s)))
step_tab <- data.frame(step_sd = steps, accept = sapply(runs, acc_rate), Rhat =
 ↪  sapply(runs, rhat))
cbind(step_sd = step_tab$step_sd, round(step_tab[c("accept", "Rhat")], 2))
```

```
  step_sd accept Rhat
1   0.005   0.93 1.78
2   0.100   0.49 1.00
3   1.500   0.04 1.01
```

```
trace_plot(runs[[1]], "step sd 0.005") + trace_plot(runs[[2]], "step sd 0.1") +
 ↪  trace_plot(runs[[3]], "step sd 1.5")
```




```
ess <- function(x, lag_max = 1000) {                    # n / (1 + 2 x sum of
 ↪  positive autocorrelations)
  rho <- acf(x, lag.max = lag_max, plot = FALSE)$acf[-1]
  k   <- which(rho <= 0)[1]; if (is.na(k)) k <- lag_max
  length(x) / (1 + 2 * sum(rho[seq_len(k - 1)]))
}
set.seed(6039)
long    <- lapply(steps, function(s) metropolis(0.6, 5000, step_sd = s))
ess_tab <- data.frame(step_sd = steps, ESS = round(sapply(long, ess)))
ess_tab
```

```
  step_sd ESS
1   0.005  10
2   0.100 993
3   1.500 119
```

```
acf_d <- do.call(rbind, lapply(seq_along(steps), function(i)
  data.frame(lag = 0:50, acf = acf(long[[i]], lag.max = 50, plot = FALSE)$acf[,
   ↪  1, 1], step = factor(steps[i]))))
ggplot(acf_d, aes(lag, acf, colour = step)) +
  geom_hline(yintercept = 0, colour = ref) + geom_line(linewidth = 0.9) +
  scale_colour_manual(values = unname(pal[1:3]), name = "step sd") +
  theme(legend.position = "right") + labs(x = "Lag", y = "Autocorrelation")
```

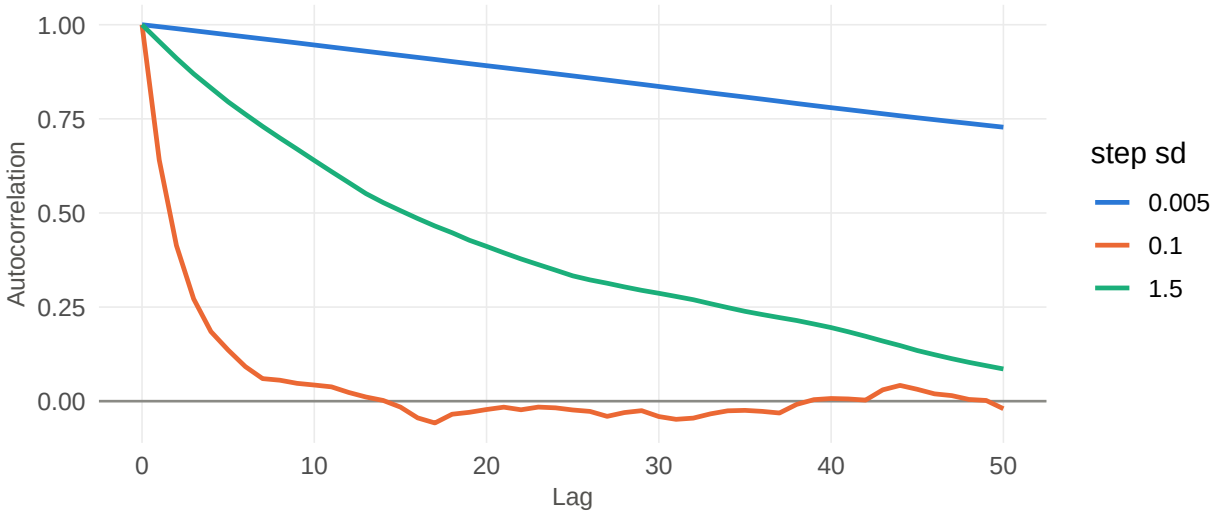


Figure 6: Autocorrelation of one long chain (5,000 draws, started inside the posterior) for each step size. Slow decay means neighbouring draws are nearly copies of each other.

3.5 *(Optional.)* Rerun the chain with the Beta(50, 50) prior by adding `a = 50, b = 50` to the call. Does the chain's mean match your hand answer to 1.2?

```
set.seed(6039)
chains50 <- sapply(starts, function(s) metropolis(s, n_iter, step_sd = 0.1, a =
 ↪  50, b = 50))
round(mean(chains50[-(1:burn), ]), 3)
```

```
[1] 0.564
```

Part 4: separation, on real data

The lecture's last example was a Hainan logit in which a single respondent aged 60+ said yes, so $\hat{\beta}_{60+} = +\infty$ and `glm()` stopped at 16.0 with a standard error of 3,956. The Chile survey has no such cell, so here is a second real dataset with the same problem. It records 384 cases filed in 1990 in which refugee claimants who had been rejected by the Canadian Immigration and Refugee Board asked the Federal Court of Appeal for leave to appeal (Greene, in Fox 2016). The outcome is whether leave was **granted**. The predictors are `rater` (1 if an independent rater judged that the case had merit) and the claimant's nation of origin. We keep the counts by nation and rater:

```
nat <- aggregate(cbind(granted = k, cases = n) ~ nation, data = greene, FUN =
 ↪  sum)
nat$rate <- nat$granted / nat$cases
knitr::kable(nat[order(nat$rate), ], digits = 2, row.names = FALSE)
```

nation	granted	cases	rate
Fiji	0	1	0.00
Poland	0	11	0.00
Bulgaria	4	36	0.11
Ghana	1	9	0.11
Nigeria	1	7	0.14
Iran	3	16	0.19
Guatemala	1	5	0.20
Lebanon	18	71	0.25
China	18	68	0.26
Somalia	8	29	0.28
Nicaragua	2	6	0.33
El.Salvador	9	26	0.35
Argentina	2	5	0.40
Sri.Lanka	26	63	0.41
Pakistan	2	4	0.50
Czechoslovakia	16	24	0.67
India	3	3	1.00

```
nat$lo   <- qbeta(0.025, 1 + nat$granted, 1 + nat$cases - nat$granted)
nat$hi   <- qbeta(0.975, 1 + nat$granted, 1 + nat$cases - nat$granted)
nat$lab  <- sprintf("%s (n = %d)", nat$nation, nat$cases)
nat$lab  <- factor(nat$lab, levels = nat$lab[order(nat$rate)])
nat$perfect <- ifelse(nat$granted == 0 | nat$granted == nat$cases, "perfect
 ↪  record", "both outcomes seen")

ggplot(nat, aes(rate, lab, colour = perfect)) +
  geom_vline(xintercept = sum(nat$granted) / sum(nat$cases), linetype = "dashed",
   ↪  colour = ref) +
  geom_linerange(aes(xmin = lo, xmax = hi), linewidth = 1.2, alpha = 0.5) +
  geom_point(aes(size = cases)) +
  scale_colour_manual(values = c(`perfect record` = pal[["orange"]], `both
   ↪  outcomes seen` = pal[["blue"]])) +
  scale_size_area(max_size = 5, breaks = c(1, 10, 30, 70), name = "cases") +
  scale_x_continuous(limits = c(0, 1), labels = scales::percent) +
  theme(legend.position = "right") +
  guides(colour = guide_legend(title = NULL, order = 1), size =
   ↪  guide_legend(order = 2)) +
  labs(x = "Share granted leave to appeal", y = NULL)
```

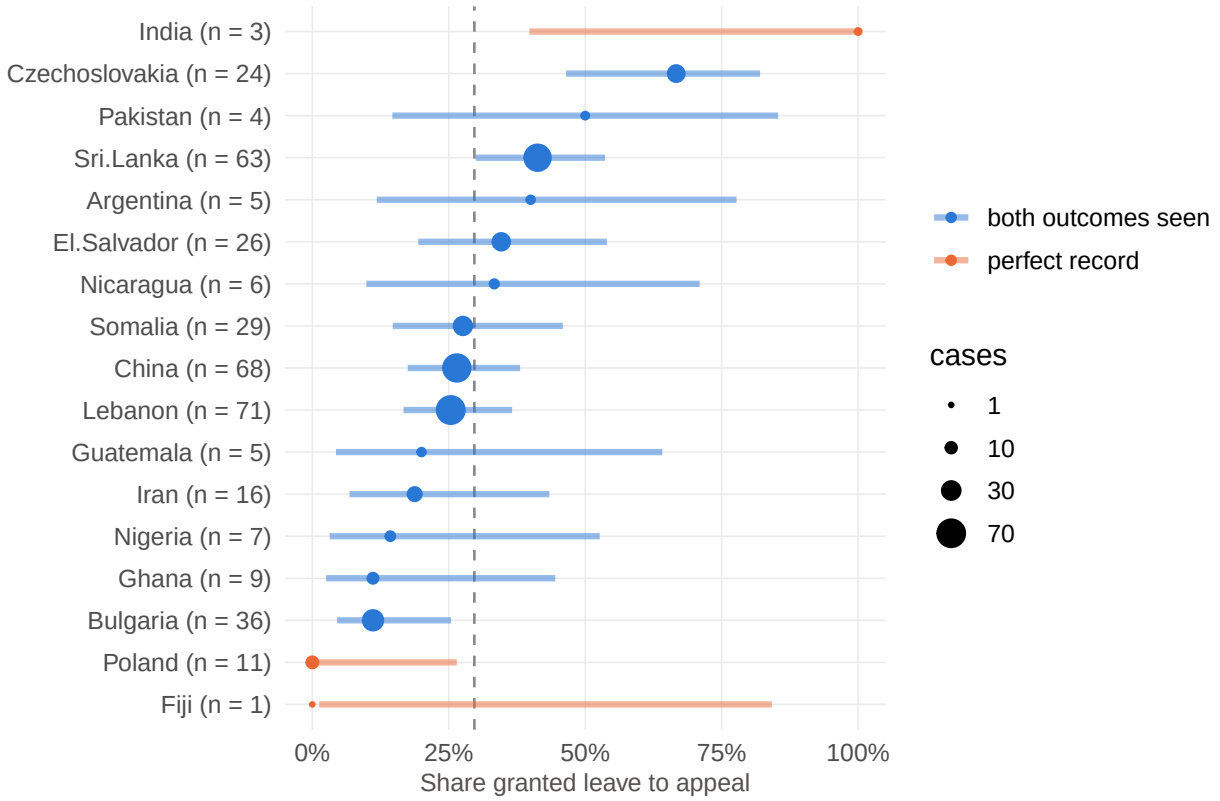


Figure 7: Grant rate by nation of origin, with 95% intervals under a flat Beta(1, 1) prior; point size shows the number of cases. Orange: nations with a perfect record. Dashed line: all nations pooled.

Three nations have perfect records: Poland (0 of 11), India (3 of 3) and Fiji (0 of 1, a single case). We give Poland and India their own indicators and pool Fiji with the other fourteen nations as the baseline. One case cannot support an indicator of its own, though a model with a separate indicator for every nation would separate on Fiji too. The data then reduce to a six-row table of counts:

```
greene$grp <- ifelse(greene$nation == "India", "India", ifelse(greene$nation ==
 ↪  "Poland", "Poland", "Other"))
cells <- aggregate(cbind(k, n) ~ rater + grp, data = greene, FUN = sum)
cells$India  <- as.numeric(cells$grp == "India")
cells$Poland <- as.numeric(cells$grp == "Poland")
cells
```

```
  rater    grp  k   n India Poland
1     0  India  1   1     1      0
2     1  India  2   2     1      0
3     0  Other 52 246     0      0
4     1  Other 59 124     0      0
5     0 Poland  0   7     0      1
```

```
6     1 Poland  0   4     0      1
```

The ordinary logit of `granted` on `rater` and the two indicators is

```
fit_glm <- glm(cbind(k, n - k) ~ rater + India + Poland, data = cells, family =
 ↪  binomial)
round(summary(fit_glm)$coefficients, 3)
```

```
            Estimate Std. Error z value Pr(>|z|)
(Intercept)   -1.317      0.156  -8.431        0
rater          1.220      0.238   5.122        0
India         25.405  81799.102   0.000        1
Poland       -24.876  65185.417   0.000        1
```

`glm()` also warns that "fitted probabilities numerically 0 or 1 occurred". The figure shows the log-likelihood as a function of β_{India} (left) and β_{Poland} (right), with the other coefficients set to their best values for each (the profile log-likelihood), relative to its supremum. A perfectly predicted group contributes nothing to the profile of the other, so each panel drops the other nation's rows.

```
sup <- as.numeric(logLik(glm(cbind(k, n - k) ~ rater, data = subset(cells, India
 ↪  == 0 & Poland == 0), family = binomial)))

prof_ll <- function(b, var, dat) {
  dat$off <- b * dat[[var]]
  as.numeric(logLik(glm(cbind(k, n - k) ~ rater + offset(off), data = dat, family
   ↪  = binomial)))
}
bi <- seq(-2, 10, by = 0.25); bp <- seq(-10, 2, by = 0.25)
d_prof <- rbind(
  data.frame(b = bi, ll = sapply(bi, prof_ll, var = "India",  dat = subset(cells,
   ↪  Poland == 0)) - sup, coef = "India"),
  data.frame(b = bp, ll = sapply(bp, prof_ll, var = "Poland", dat = subset(cells,
   ↪  India == 0))  - sup, coef = "Poland"))

ggplot(d_prof, aes(b, ll)) +
  geom_hline(yintercept = 0, linetype = "dashed", colour = ref) +
  geom_line(colour = pal[["blue"]], linewidth = 0.9) +
  facet_wrap(~ coef, scales = "free_x") +
  scale_x_continuous(n.breaks = 5) +
  theme(panel.spacing.x = unit(1.6, "lines")) +
  labs(x = "Coefficient", y = "Profile log-likelihood minus supremum")
```

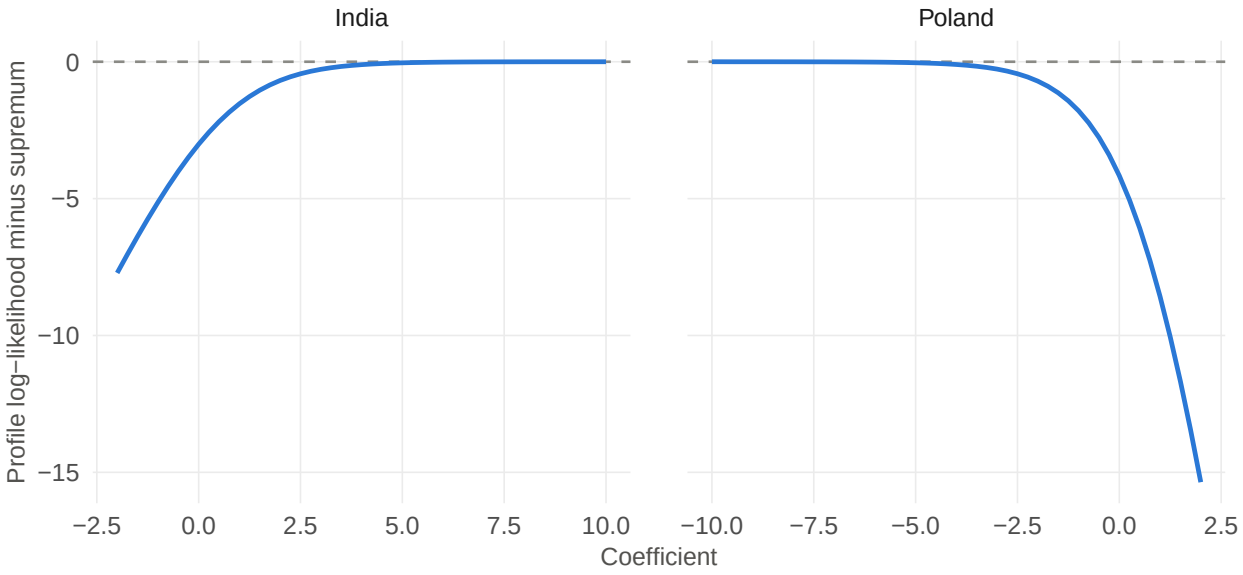


Figure 8: Profile log-likelihood of the India (left) and Poland (right) coefficients, minus the supremum. Each climbs toward the dashed line and never turns down.

4.1 What do you notice in the table of nations? Use the figure to explain why the MLEs of β_{India} and β_{Poland} do not exist, and say what `glm()` is reporting instead.

4.2 `glm()` reports $p = 1.00$ for Poland and $p = 1.00$ for India. Does that mean the nation indicators are unrelated to whether leave to appeal is granted? Compare with the likelihood-ratio tests from `drop1()`.

```
lrt <- drop1(fit_glm, test = "LRT")
lrt
```

```
Single term deletions

Model:
cbind(k, n - k) ~ rater + India + Poland
       Df Deviance    AIC     LRT Pr(>Chi)
<none>      0.0000 18.829
rater   1  26.6647 43.493 26.6647 2.42e-07 ***
India   1   6.0291 22.858  6.0291 0.014072 *
Poland  1   8.3259 25.155  8.3259 0.003908 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

4.3 We put an independent $N(0, 2.5^2)$ prior on all four coefficients (intercept included), as on slide 22. **(a)** What does this say about the odds ratio for Poland before seeing data? **(b)** Why did we not need to standardize any predictor first, as we would for a continuous predictor?

The sampler is the same algorithm, now moving all four coefficients at once. Because the data are a six-row table of counts, the log-likelihood is a sum over the six rows. The log posterior is the log-likelihood plus the log prior, where the prior sd for the two nation coefficients is an argument (it is 2.5 for now):

```
X <- cbind(Intercept = 1, rater = cells$rater, India = cells$India, Poland =
 ↪  cells$Poland)

log_lik <- function(beta) {
  eta <- drop(X %*% beta)
  sum(cells$k * plogis(eta, log.p = TRUE) + (cells$n - cells$k) * plogis(-eta,
   ↪  log.p = TRUE))
}
make_log_post <- function(nat_sd) {
  prior_sd <- c(Intercept = 2.5, rater = 2.5, India = nat_sd, Poland = nat_sd)
  function(beta) log_lik(beta) + sum(dnorm(beta, 0, prior_sd, log = TRUE))
}

rwm <- function(start, n_iter, step, log_post) {
  out <- matrix(NA_real_, n_iter, length(start), dimnames = list(NULL,
 ↪  names(start)))
  cur <- start; lp_cur <- log_post(cur)
  for (t in seq_len(n_iter)) {
    prop <- cur + rnorm(length(cur), 0, step)                    # a step in
 ↪  every coefficient at once
    lp_prop <- log_post(prop)
    if (log(runif(1)) < lp_prop - lp_cur) { cur <- prop; lp_cur <- lp_prop }
    out[t, ] <- cur
  }
  out
}

fit_mcmc <- function(nat_sd, n_iter = 20000, burn = 2000, n_chains = 4) {
  step   <- c(Intercept = 0.3, rater = 0.45, India = 1.5, Poland = 1.5)    #
 ↪  tuned in a pilot run (see 3.4)
  chains <- lapply(seq_len(n_chains), function(j)
    rwm(setNames(rnorm(4), colnames(X)), n_iter, step,
     ↪  make_log_post(nat_sd))[-(1:burn), ])
  list(draws  = do.call(rbind, chains),
       rhat   = sapply(colnames(X), function(k) rhat(sapply(chains, function(m)
        ↪  m[, k]))),
       accept = mean(sapply(chains, function(m) mean(diff(m[, "India"]) != 0))))
}
post_summary <- function(fit) {
  d <- fit$draws
  rbind(mean = colMeans(d), sd = apply(d, 2, sd), `2.5%` = apply(d, 2, quantile,
   ↪  0.025),
```

```
        `97.5%` = apply(d, 2, quantile, 0.975), Rhat = fit$rhat)
}
```

```
set.seed(6039)
weak <- fit_mcmc(2.5)
knitr::kable(round(t(post_summary(weak)), 2))
```

	mean	sd	2.5%	97.5%	Rhat
Intercept	-1.31	0.15	-1.63	-1.02	1
rater	1.21	0.24	0.75	1.67	1
India	2.92	1.54	0.26	6.29	1
Poland	-3.14	1.48	-6.40	-0.64	1

```
c(`P(beta_India > 0)` = mean(weak$draws[, "India"] > 0), `P(beta_Poland < 0)` =
 ↪  mean(weak$draws[, "Poland"] < 0),
  accept = weak$accept)
```

```
 P(beta_India > 0) P(beta_Poland < 0)            accept
         0.9853889         0.9962639         0.1689261
```

```
have_brglm2 <- requireNamespace("brglm2", quietly = TRUE)
fit_firth <- if (have_brglm2) glm(cbind(k, n - k) ~ rater + India + Poland, data
 ↪  = cells, family = binomial,
                                  method = brglm2::brglmFit, type = "AS_mean")
if (have_brglm2) round(summary(fit_firth)$coefficients, 2)      # Firth's
 ↪  penalized likelihood, for comparison
```

```
            Estimate Std. Error z value Pr(>|z|)
(Intercept)    -1.31       0.16   -8.41     0.00
rater           1.21       0.24    5.11     0.00
India           2.64       1.83    1.44     0.15
Poland         -2.45       1.54   -1.59     0.11
```

4.4 What does the posterior say about β_{India} and β_{Poland}? Are they finite? Compare with Firth's penalized-likelihood estimates (the last table).

4.5 The prior matters here. We vary the prior sd of the two nation coefficients over three values: 2.5 (the weak default), 10 (almost flat), and a prior built from the other nations, as in the lecture's "priors from other provinces". The seven nations with at least 20 cases have grant-rate log-odds with the standard deviation printed below; we use it as the sd for how far a nation can sit from the typical nation. Run the sampler for each and compare. Which estimates are stable and which are not? What does that tell you about what the data can say?

```
set.seed(6039)
big        <- subset(nat, cases >= 20)                    # the seven nations
 ↪  with at least 20 cases
sd_nations <- sd(qlogis(big$granted / big$cases))
sd_nations
```

```
[1] 0.8410793
```

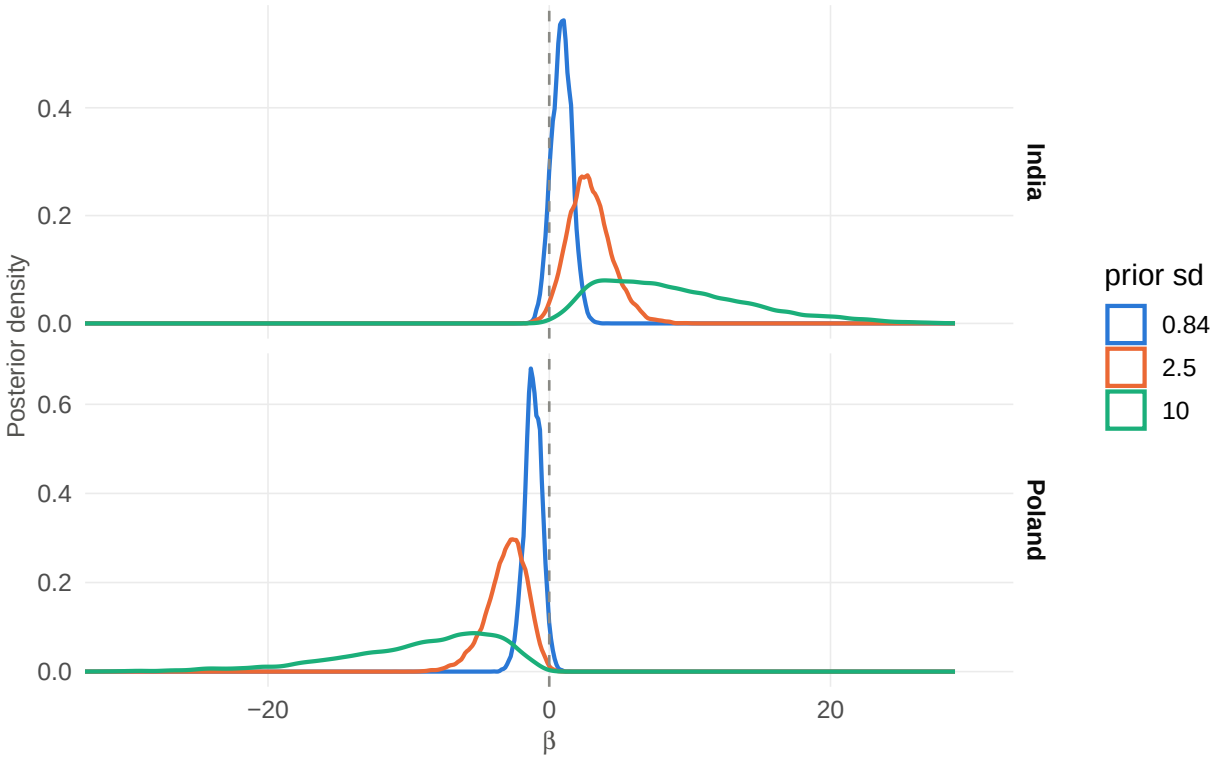
```
prior_sds <- c(other = sd_nations, weak = 2.5, wide = 10)
sens      <- lapply(prior_sds, fit_mcmc)

sens_tab <- do.call(rbind, lapply(seq_along(prior_sds), function(i) {
  d <- sens[[i]]$draws
  data.frame(prior_sd = prior_sds[i],
             India_mean = mean(d[, "India"]), India_hi = quantile(d[, "India"],
              ↪  0.975), P_India_pos = mean(d[, "India"] > 0),
             Poland_mean = mean(d[, "Poland"]), Poland_lo = quantile(d[,
              ↪  "Poland"], 0.025), P_Poland_neg = mean(d[, "Poland"] < 0),
             rater_mean = mean(d[, "rater"]), max_Rhat = max(sens[[i]]$rhat),
              ↪  row.names = NULL)
}))
knitr::kable(sens_tab, digits = c(2, 2, 1, 3, 2, 1, 3, 2, 3),
             col.names = c("Prior sd", "India mean", "India 97.5%", "P(India >
              ↪  0)", "Poland mean",
                           "Poland 2.5%", "P(Poland < 0)", "rater mean", "max
                            ↪  Rhat"))
```

Prior sd	India mean	India 97.5%	P(India > 0)	Poland mean	Poland 2.5%	P(Poland < 0)	rater mean	max Rhat
0.84	0.89	2.3	0.891	-1.15	-2.5	0.974	1.20	1.001
2.50	2.88	6.3	0.984	-3.03	-6.3	0.997	1.21	1.003
10.00	8.92	21.6	0.998	-9.15	-23.3	1.000	1.21	1.009

```
dens <- do.call(rbind, lapply(seq_along(prior_sds), function(i) {
  d <- sens[[i]]$draws
  rbind(data.frame(b = d[, "India"],  coef = "India",  prior =
   ↪  factor(round(prior_sds[i], 2))),
        data.frame(b = d[, "Poland"], coef = "Poland", prior =
         ↪  factor(round(prior_sds[i], 2))))
}))
ggplot(dens, aes(b, colour = prior)) + geom_density(linewidth = 0.8) +
  geom_vline(xintercept = 0, colour = ref, linetype = "dashed") +
  facet_grid(coef ~ ., scales = "free_y") +
  scale_colour_manual(values = unname(pal[1:3]), name = "prior sd") +
```

```
  coord_cartesian(xlim = c(-30, 30)) + theme(legend.position = "right",
   ↪  strip.text = element_text(colour = ink, face = "bold")) +
  labs(x = expression(beta), y = "Posterior density")
```



Part 5: the real result, exit ticket and Thursday

5.1 Put all 1,757 decided respondents together with a flat Beta(1, 1) prior and compute the national posterior for the Yes share. The official result of the 5 October 1988 plebiscite was 44.01% Yes. Does the 95% interval cover it? If not, name two reasons the survey could be off that the posterior cannot see.

```
Y_all <- sum(chile$Y); N_all <- sum(chile$N); U_all <- sum(chile$U)
nat_post <- c(mean = (1 + Y_all) / (2 + Y_all + N_all),
              lo = qbeta(0.025, 1 + Y_all, 1 + N_all), hi = qbeta(0.975, 1 +
               ↪  Y_all, 1 + N_all))
round(nat_post, 3)
```

```
 mean    lo    hi
0.494 0.471 0.517
```

```
px  <- seq(0.42, 0.56, length.out = 400)
p_nat <- ggplot(data.frame(p = px, d = dbeta(px, 1 + Y_all, 1 + N_all)), aes(p,
 ↪  d)) +
  geom_area(data = ~ subset(.x, p >= nat_post["lo"] & p <= nat_post["hi"]), fill
   ↪  = pal[["blue"]], alpha = 0.25) +
  geom_line(colour = pal[["blue"]], linewidth = 0.9) +
  labs(x = "National share saying Yes (decided respondents)", y = "Posterior
   ↪  density")
p_nat
```



Figure 9: Posterior for the national Yes share under a flat Beta(1, 1) prior, with its central 95% interval shaded.

5.2 In two sentences: what did the prior do in the estimate for region M (56 respondents), and what did it do in the India and Poland coefficients (separation)?

5.3 Slide 25 says data from other provinces count only as far as provinces are alike, and a hierarchical model learns how alike from the data. In Part 1, what did we have to *assume* that such a model would instead *estimate*? Here is what one fixed prior, Beta(20, 20), does to all five regions at once:

```
shr <- chile
shr$post <- (20 + shr$Y) / (40 + shr$decided)
shr$lab  <- factor(sprintf("%s (n = %d)", shr$region, shr$decided), levels =
 ↪  sprintf("%s (n = %d)", shr$region, shr$decided)[order(shr$decided)])

ggplot(shr, aes(y = lab)) +
  geom_vline(xintercept = 0.5, linetype = "dashed", colour = ref) +
  geom_segment(aes(x = share, xend = post, yend = lab), colour = ref, linewidth =
   ↪  0.8,
               arrow = arrow(length = unit(0.08, "in"), type = "open")) +
```

```
  geom_point(aes(x = share, colour = "Sample share"), size = 3.4) +
  geom_point(aes(x = post, colour = "Posterior mean"), size = 3.4) +
  scale_colour_manual(values = c(`Sample share` = pal[["orange"]], `Posterior
   ↪  mean` = pal[["blue"]]), name = NULL) +
  scale_x_continuous(breaks = seq(0.4, 0.7, 0.1)) +
  theme(legend.position = "bottom") +
  labs(x = "Share saying Yes", y = NULL)
```

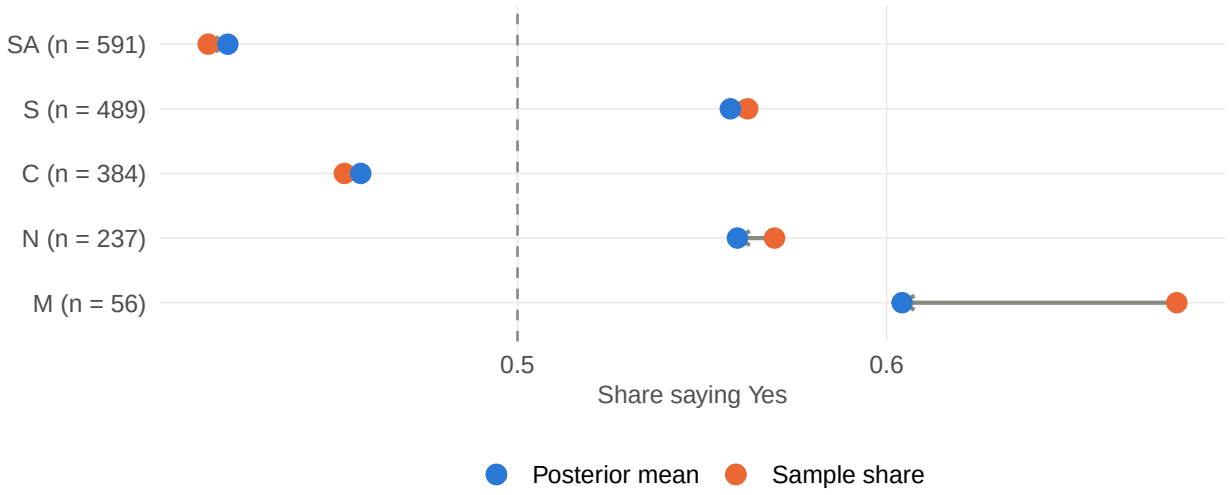


Figure 10: Sample share (orange) and posterior mean (blue) for each region under the same Beta(20, 20) prior. The small regions move most. The dashed line is the prior mean.

Part 6: take-home, a Stan check

Slide 24's packages fit the same model without our sampler. This part needs `rstanarm`, which ships a precompiled sampler for `stan_glm()`.

```
library(rstanarm)
fit_stan <- stan_glm(cbind(k, n - k) ~ rater + India + Poland, data = cells,
 ↪  family = binomial,
                     prior = normal(0, 2.5, autoscale = FALSE),
                     prior_intercept = normal(0, 2.5, autoscale = FALSE),
                     chains = 4, iter = 4000, seed = 6039, refresh = 0)
stan_tab <- summary(fit_stan, probs = c(0.025, 0.975))[1:4, c("mean", "sd",
 ↪  "2.5%", "97.5%", "Rhat")]
round(stan_tab, 2)
```

```
             mean   sd  2.5% 97.5% Rhat
(Intercept) -1.32 0.15 -1.62 -1.02    1
```

```
rater        1.21 0.24  0.73  1.67    1
India        3.02 1.59  0.27  6.45    1
Poland      -3.01 1.42 -6.21 -0.62    1
```

6.1 Compare these numbers with `weak` in 4.4. Do they agree? Should they?

6.2 Gelman et al. (2008) recommend a heavy-tailed default, a Cauchy(0, 2.5) prior on the coefficients. *Predict:* will the lower end of Poland's interval go up or down relative to the normal prior? Then replace `normal(0, 2.5, ...)` by `cauchy(0, 2.5, ...)` and check.

```
fit_cauchy <- stan_glm(cbind(k, n - k) ~ rater + India + Poland, data = cells,
 ↪  family = binomial,
                       prior = cauchy(0, 2.5, autoscale = FALSE),
                       prior_intercept = cauchy(0, 10, autoscale = FALSE),
                       chains = 4, iter = 4000, seed = 6039, refresh = 0)
cauchy_tab <- summary(fit_cauchy, probs = c(0.025, 0.975))[1:4, c("mean", "sd",
 ↪  "2.5%", "97.5%", "Rhat")]
round(cauchy_tab, 2)
```

```
             mean   sd   2.5% 97.5% Rhat
(Intercept) -1.31 0.16  -1.62 -1.01 1.00
rater        1.21 0.24   0.76  1.67 1.00
India        6.77 6.69   0.44 28.76 1.01
Poland      -6.59 6.23 -25.12 -0.85 1.01
```

References

Efron, B. & Hastie, T. (2016) *Computer Age Statistical Inference*, ch. 3. Cambridge University Press.

Fox, J. (2016) *Applied Regression Analysis and Generalized Linear Models*, 3rd ed. Sage. (Chile and Greene data, distributed in the `carData` package. Chile: survey by FLACSO/Chile, April-May 1988; Greene: refugee-appeal cases of 1990, from Ian Greene, York University.)

Gelman, A., Jakulin, A., Pittau, M. G. & Su, Y.-S. (2008) "A Weakly Informative Default Prior Distribution for Logistic and Other Regression Models." *Annals of Applied Statistics* 2(4): 1360-1383. doi:10.1214/08-AOAS191

Heinze, G. & Schemper, M. (2002) "A Solution to the Problem of Separation in Logistic Regression." *Statistics in Medicine* 21(16): 2409-2419. doi:10.1002/sim.1047

Johnson, A. A., Ott, M. Q. & Dogucu, M. (2022) *Bayes Rules! An Introduction to Applied Bayesian Modeling*, ch. 6. https://www.bayesrulesbook.com

Kosmidis, I. & Firth, D. (2021) "Jeffreys-Prior Penalty, Finiteness and Shrinkage in Binomial-Response Generalized Linear Models." *Biometrika* 108(1): 71-82. doi:10.1093/biomet/asaa052

Kuran, T. (1995) *Private Truths, Public Lies: The Social Consequences of Preference Falsification.* Harvard University Press.

Rainey, C. (2016) "Dealing with Separation in Logistic Regression Models." *Political Analysis* 24(3): 339-355. doi:10.1093/pan/mpw014

Wikipedia, "1988 Chilean presidential referendum" (official result: 44.01% Yes, 55.99% No; vote held 5 October 1988) and "1980 Chilean constitutional referendum" (official result and reported irregularities: no electoral roll, no polling-station monitors for the opposition). https://en.wikipedia.org/wiki/1988_Chilean_presidential_referendum, https://en.wikipedia.org/wiki/1980_Chilean_constitutional_referendum